

18.1 Priority Queues

Binary Search Trees allowed us to efficiently search trees, only taking $\log(N)$ time. This was because we could eliminate half of the elements of the tree at every step of our search. What if we cared more about quickly finding the *smallest* or *largest* element instead of quickly searching?

Now we come to the Abstract Data Type of a *Priority Queue*. To understand this ADT, consider a bag of items. You can add items to this bag, you can remove items from this bag, etc. The one caveat is that you can only interact with the smallest items of this bag.

```
/** (Min) Priority Queue: Allowing tracking and removal of
 * the smallest item in a priority queue. */
public interface MinPQ<Item> {
    /** Adds the item to the priority queue. */
    public void add(Item x);
    /** Returns the smallest item in the priority queue. */
    public Item getSmallest();
    /** Removes the smallest item from the priority queue. */
    public Item removeSmallest();
    /** Returns the size of the priority queue. */
    public int size();
}
```

Using Priority

Where would we actually use this structure or need it?

- Consider the scenario where we are monitoring text messages between citizens and want to keep track of unharmonious conversations.
- Each day, you prepare a report of M messages that are the most unharmonious using a `HarmoniousnessComparator`.

Let's take this approach: Collect all of the messages that we receive throughout the day into a single list. Sort this list and return the first M messages.

```
public List<String> unharmoniousTexts(Sniffer sniffer, int M) {
    ArrayList<String> allMessages = new ArrayList<String>();
    for (Timer timer = new Timer(); timer.hours() < 24; ) {
        allMessages.add(sniffer.getNextMessage());
    }

    Comparator<String> cmptr = new HarmoniousnessComparator();
    Collections.sort(allMessages, cmptr, Collections.reverseOrder());

    return allMessages.sublist(0, M);
}
```

Potential downsides? This approach will use $\Theta(N)$ space when actually we only really need to use $\Theta(M)$ space.

Exercise 13.1.1. Complete the method listed above, `unharmoniousTexts` with the same functionality as described using only $\Theta(M)$ space.

Priority Queue Implementation

We solved the same problem using the Priority Queue ADT, making memory more efficient. We can observe that the code is slightly more complicated, but this is not always the case.

Remember that ADT are similar interfaces and the implementation is still to be defined. Let's consider possible implementations using the data structure implementations we already know, analyzing the **worst case** runtimes of our desired operations:

- Ordered Array
 - add: $\Theta(N)$
 - getSmallest: $\Theta(1)$
 - removeSmallest: $\Theta(1)$
- Bushy BST
 - add: $\Theta(\log N)$
 - getSmallest: $\Theta(\log N)$
 - removeSmallest: $\Theta(\log N)$
- HashTable
 - add: $\Theta(1)$
 - getSmallest: $\Theta(N)$
 - removeSmallest: $\Theta(N)$

Exercise 13.1.2. Explain each of the runtimes above. What are the downfalls of each data structure? Describe a way of modifying one of these data structures to improve performance.

Can we do better than these suggested data structures?

Summary

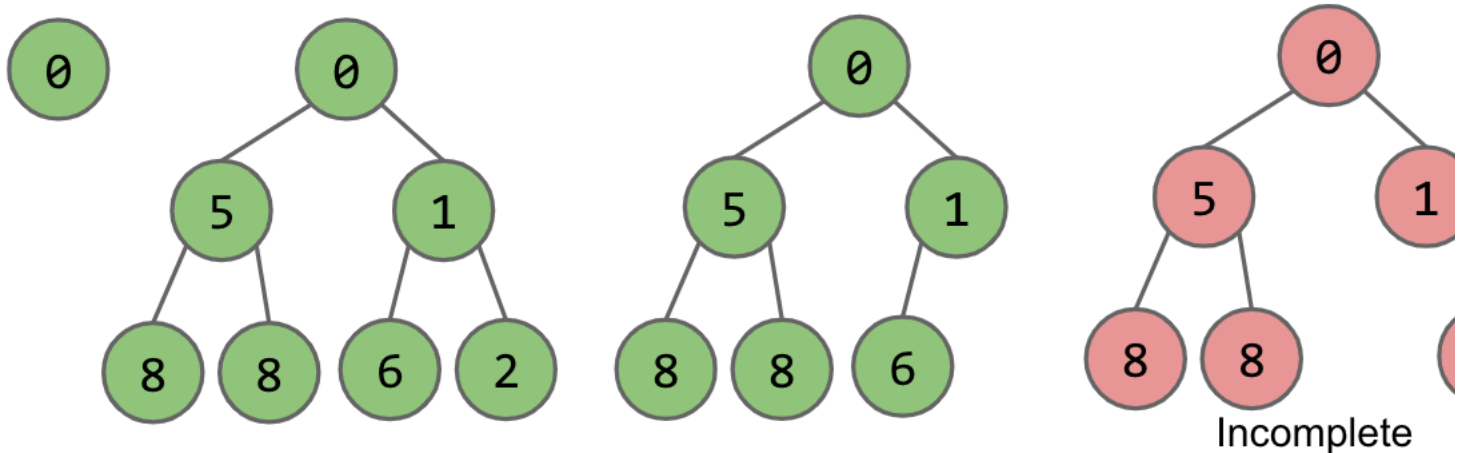
- Priority Queue is an Abstract Data Type that optimizes for handling minimum or maximum elements.
- There can be space/memory benefits to using this specialized data structure.
- Implementations for ADTs that we currently know don't give us efficient runtimes for PQ operations.
 - A binary search tree is the most efficient among the other structures

18.2 Heaps

We previously saw that our known data structures with the best runtime for our PQ operations was the *binary search tree*. Modifying its structure and the constraints, we can further improve the runtime and efficiency of these operations.

We will define our binary min-heap as being **complete** and obeying **min-heap** property:

- Min-heap: Every node is less than or equal to both of its children
- Complete: Missing items only at the bottom level (if any), all nodes are as far left as possible.



As we can see in the figures above, the green colored heaps are valid and the red ones aren't. The last two aren't because they violate at least one of the properties that we defined above.

Now let's consider how this structure lends itself to the abstract data type we described in the previous chapter. We will do this through analyzing our desired operations.

Exercise 13.2.1. Determine how each method of our Priority Queue interface will be implemented given this heap structure. Don't write actual code, just pseudocode!

Heap Operations

The three methods we care about for the PriorityQueue ADT are `add`, `getSmallest`, and `removeSmallest`. We will start by conceptually describing how these methods can be implemented given our given schema of a heap.

- `add`: Add to the end of heap temporarily. Swim up the hierarchy to the proper place.
 - Swimming involves swapping nodes if `child < parent`
- `getSmallest`: Return the root of the heap (This is guaranteed to be the minimum by our *min-heap* property)
- `removeSmallest`: Swap the last item in the heap into the root. Sink down the hierarchy to the proper place.
 - Sinking involves swapping nodes if `parent > child`. Swap with the smallest child to preserve *min-heap* property.

Great! We have determined how we will approach the operations specified by the PriorityQueue interface in an efficient way. But how do we actually code this?

Exercise 13.2.2. Give the runtime for each of the methods specified above. Worst cases and best cases.

Tree Representation

There are many approaches we can take to representing trees.

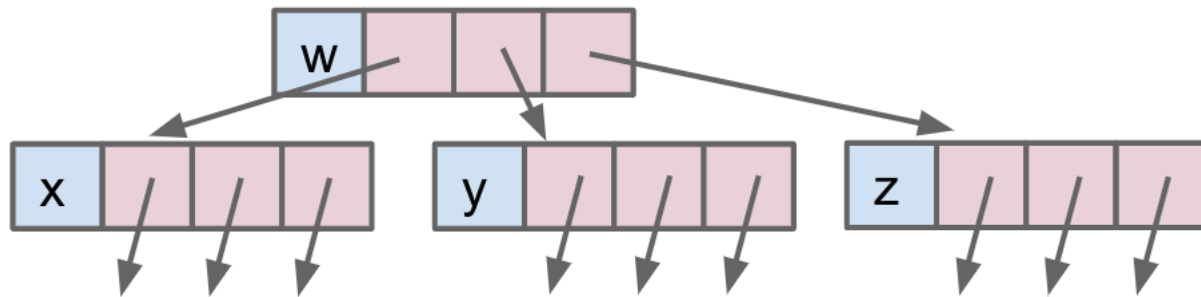
Approach 1a, 1b, and 1c

Let us consider the most intuitive and previously used representation for trees. We will create mappings between nodes and their children. There are several ways to do this which we will explore right now.

- In approach **Tree1A**, we consider creating pointers to our children and storing the value inside of the node object. These are hardwired links that give us fixed-width nodes. We can observe the code:

```
public class Tree1A<Key> {
    Key k;
    Tree1A left;
    Tree1A middle;
    Tree1A right;
    ...
}
```

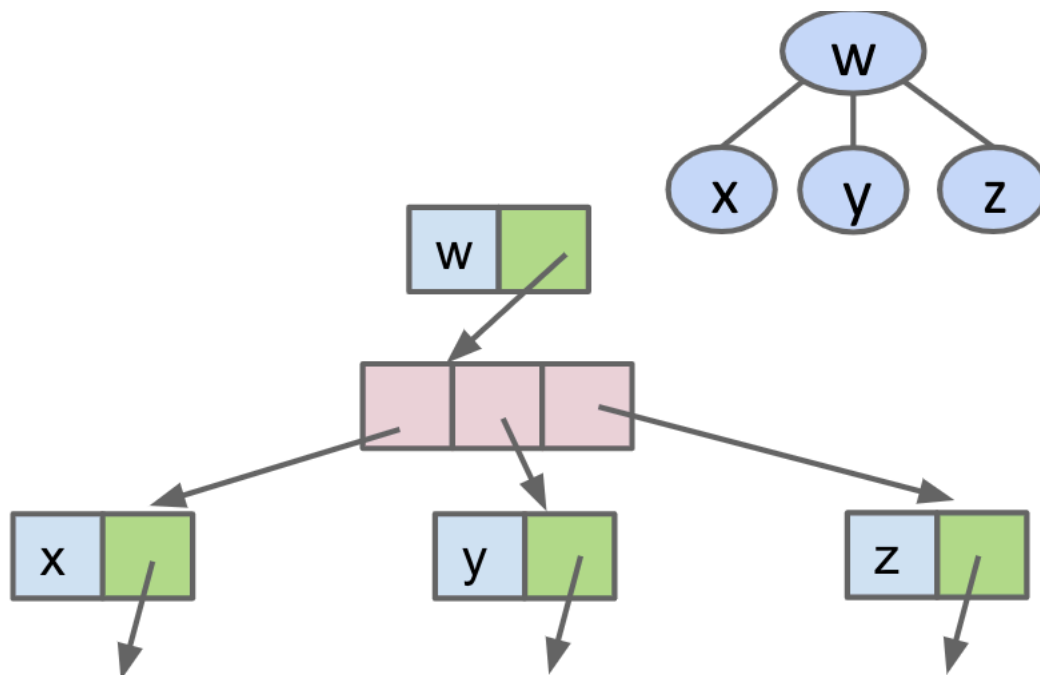
The visualization of this type of structure is shown below.



1a: Fixed-Width Nodes (BSTMap used this approach)

- Alternatively, in **Tree1B**, we explore the use of arrays as representing the mapping between children and nodes. This would give us variable-width nodes, but also awkward traversals and performance will be worse.

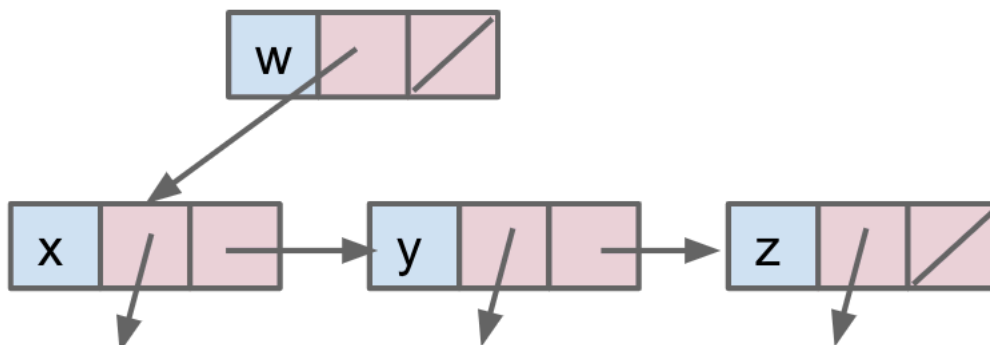
```
public class Tree1B<Key> {
    Key k;
    Tree1B[] children;
    ...
}
```



1b: Variable-Width Nodes

- Lastly, we can use the approach for **Tree1C**. This will be slightly different from the usual approaches that we've seen. Instead of only representing a node's children, we say that nodes can also maintain a reference to their siblings.

```
public class Tree1C<Key> {
    Key k;
    Tree1C favoredChild;
    Tree1C sibling;
    ...
}
```



1c: Sibling Tree

In all of these approaches, we store explicit references to who is below us. These explicit references take the form of pointers to the actual Tree objects that are our children. Let's think of more exotic approaches that don't store explicit references to children.

Approach 2

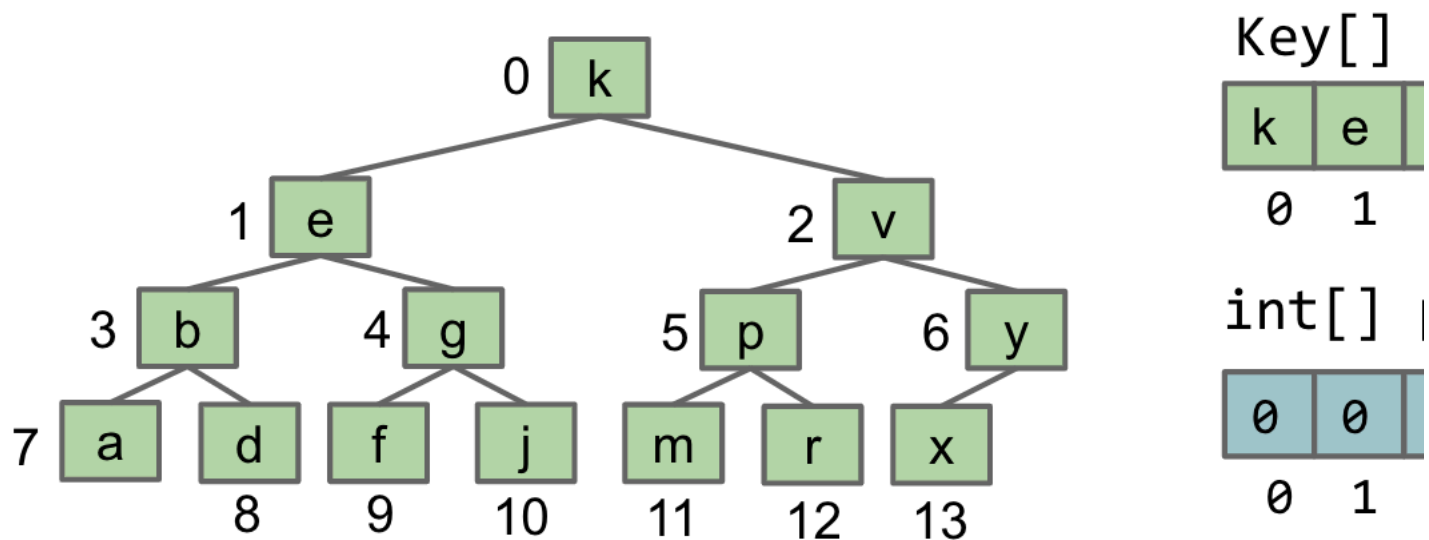
Recall the Disjoint Sets ADT. The way that we represented this Weighted Quick Union structure was through arrays. For representing a tree, we can store the *keys* array as well as a *parents* array. The keys array represent which index maps to which key, and the parents array represents which key is a child of another key.

```
public class Tree2<Key> {
    Key[] keys;
    int[] parents;
    ...
}
```

Take some time to ensure that the tree on the left corresponds to the representation in the arrays on the right.

It's time to make a very important observation! Based on the structure of the tree and the relationship between the array representations and the diagram of the tree, we can see:

1. The tree is **complete**. This is a property we have defined earlier.
2. The parents array has a sort of redundant pattern where elements are just doubled.
3. Reading the level-order of the tree, we see that it matches exactly the order of the keys in the *keys* array.

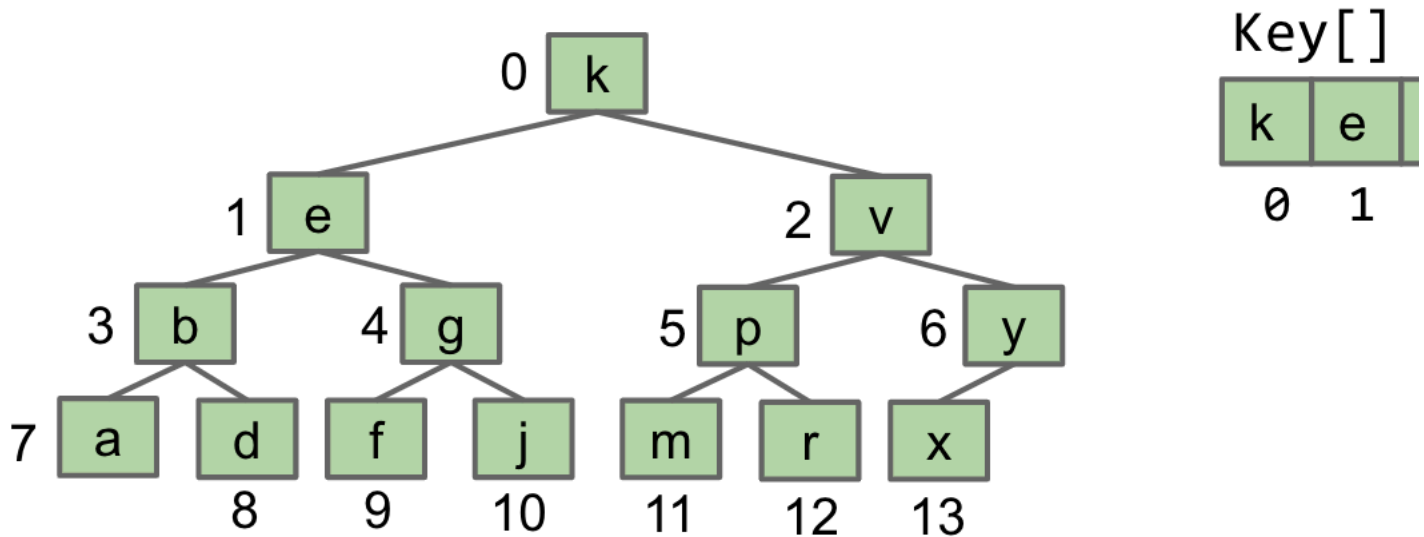
**Approach 2**

What does this all mean? We know the parents array is redundant so we can ignore it and we know that a tree can be represented by level order in an array.

Approach 3

In this approach, we assume that our tree is complete. This is to ensure that there are no “gaps” inside of our array representation. Thus, we will take this complex 2D structure of the tree and flatten it into an array.

```
public class TreeC<Key> {
    Key[] keys;
    ...
}
```



Swim

Given this implementation, we define the following code for the “swim” described in the Heap Operations section.

```

public void swim(int k) {
    if (keys[parent(k)] > keys[k]) {
        swap(k, parent(k));
        swim(parent(k));
    }
}

```

What does the parent method do? It returns the parent of the given k using the representation in Approach 3.

Exercise 13.2.3. Write the parent method. For extra challenge, try to write the methods for finding the left child and right child of a given item.

18.3 PQ Implementation

The actual implementation, which we will use and the book uses, is quite similar to the representation we discussed at the end of the previous chapter. The one difference is that we will leave one empty spot at the beginning of the array to simplify computation.

- $\text{leftChild}(k) = k * 2$
- $\text{rightChild}(k) = k * 2 + 1$
- $\text{parent}(k) = k/2$

Comparing to alternative implementations

Methods	Ordered Array	Bushy BST	Hash Table	Heap
add	$\Theta(N)$	$\Theta(\log N)$	$\Theta(1)$	$\Theta(\log N)$
getSmallest	$\Theta(1)$	$\Theta(\log N)$	$\Theta(N)$	$\Theta(1)$
removeSmallest	$\Theta(N)$	$\Theta(\log N)$	$\Theta(N)$	$\Theta(\log N)$

Awesome! We can see that we have improved our runtime and we have also solved the problem of duplicate elements. Couple notes:

- Heap operations are **amortized** analysis, since the array will have to resize (not a big deal)
- BST's can have constant time `getSmallest` if pointer is stored to smallest element
- Array-based heaps take around 1/3rd the memory it takes to represent a heap using approach 1A (direct pointers to children)

Exercise 13.3.1 Some lingering implementation questions:

1. How will the PQ know how to order the items? Say we had a PQ of dogs, would it order by weight or breed?
2. Is there a way to allow for a flexibility of orderings?
3. What could we do to make a MinPQ into a MaxPQ?

18.4 Summary

Priority Queue. A Max Priority Queue (or PQ for short) is an ADT that supports at least the insert and delete-max operations. A MinPQ supports insert and delete-min.

Heaps. A max (min) heap is an array representation of a binary tree such that every node is larger (smaller) than all of its children. This definition naturally applies recursively, i.e. a heap of height 5 is composed of two heaps of height 4 plus a parent.

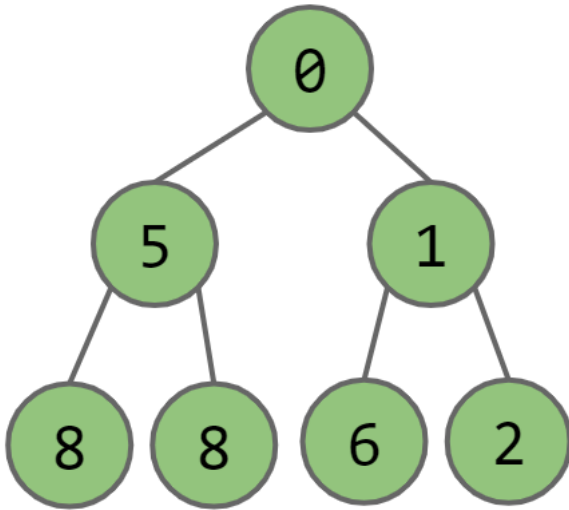
Tree Representations. Know that there are many ways to represent a tree, and that we use Approach 3b (see lecture slides) for representing heaps, since we know they are complete.

Running times of various PQ implementations. Know the running time of the three primary PQ operations for an unordered array, ordered array, and heap implementation.

18.5 Exercises

Procedural

1. Represent the following heap as an array.



2. Consider the above heap. What range of values, when inserted, will cause the maximum number of swaps?

- Problem 1
- Problem 2

Metacognitive

1. Consider an externally-chained hashmap. If there are many hash collisions, what data structure would be most efficient in storing the key-value pairs at each bucket?
2. Consider an array sorted in descending order. Is this a valid heap? If so, which type: min-heap, or max-heap?
3. Suppose that we store the root of a heap at the 0 th index in our array. For an arbitrary index k , compute the indices of its children and parent.
4. Explain how you would implement a stack using a priority queue.

- Problem 1
- Problem 2
- Problem 3
- Problem 4

18. Heaps and Priority Queues